STATIC APPLICATION SECURITY REVIEW

Security Assessment

inventoryApp — hand-receipt / property-book system · revision `ff4857f` · 2026-08-05

Application: inventoryApp — hand-receipt / property-book system (Next.js 16 App Router, React 19, TypeScript 5, Prisma 7 / PostgreSQL, Auth.js v5)
Revision assessed: `ff4857fd363fa548b7d639a7afd95b3c1d8363ad` (branch `feat/receipt-link-pin-bypass`)
Date: 2026-08-05
Method: Static, read-only multi-agent source review (Claude Security plugin) — inventory, threat model, 12 parallel researchers across component × lens cells, a three-lens verification panel (reachability / impact / defenses, 2-of-3 to survive), and an adversarial refutation pass against every survivor.
Working tree state: Only `package.json` / `package-lock.json` modified (one devDependency addition, `@testing-library/jest-dom` — not runtime-relevant).

> **Nothing was executed.** No build, no test run, no running server, no HTTP requests, and no live database or deployed-environment access. Every conclusion below is derived from reading application source and installed dependency source. Section 10 states precisely what this leaves unverified.

1. Executive summary

This codebase is **well secured, deliberately so**, and the security work is unusually disciplined for its size. The authorization model is the strongest part: all 49 Server Actions and all 6 Route Handlers gate themselves as their first awaited statement, all 10 admin pages call `requireAdmin()` directly rather than inheriting a gate from a layout or the proxy, and `requireUser()` / `requireAdmin()` re-read both `role` and `isActive` from the database on every request and fail closed. No action derives identity, role, or signature material from client input.

The dangerous-sink sweep came back clean: **no SQL injection, no command injection, no `eval`, no path traversal, no SSRF, no open redirect, no unsafe deserialization, no prototype pollution, and no `dangerouslySetInnerHTML` anywhere in the tree.** The single place a caller-influenced identifier reaches SQL is guarded twice with `Object.hasOwn`. The newest and most security-sensitive control — the per-receipt capability token that bypasses the public PIN gate — was scrutinized specifically for crypto and scope defects and **survived every attack constructed against it** (domain separation, constant-time compare, exact-match receipt lookup, anchored path regex, and correct refusal to widen `publicAccessAllowed()`).

What survived verification is a short tail of five findings — **none Critical, none High**. Three are Medium and two are Low.

The dominant risk theme is not missing controls. It is narrow gaps between what the documentation asserts and what the code does. The most consequential finding (F1) exists in exactly that gap: `docs/SECURITY.md:57` states that a password change revokes existing sessions, and for the self-service path it does not. That class of defect is dangerous out of proportion to its severity, because it misleads an incident responder at the moment they are relying on the document. For that reason the documentation reconciliation in section 9 should be read as a first-class result of this assessment, not an appendix.

Severity	Count
Critical	0
High	0

Severity	Count
Medium	3
Low	2
Total verified	**5**

Additionally: **16 accepted risks** are enumerated and adjudicated (section 8), **21 candidate findings** were checked and cleared by verification (section 11), and **11 undocumented risks** were identified that appear in the code but not in the project's own risk register (section 9.3).

---

2. Baselines and framing

Findings are classified against three references, plus the three lenses this assessment was asked to apply.

- **OWASP Top 10 (2021)** — category assignment per finding.
- **CWE** — specific weakness identification (MITRE Common Weakness Enumeration).
- **NIST SP 800-53 rev. 5** control families, used loosely for the mitigation grouping: `AC` (Access Control), `IA` (Identification & Authentication), `SI` (System & Information Integrity), `AU` (Audit & Accountability), `SC` (System & Communications Protection).

The three assessment lenses

Lens	Question it asks	Verdict
Principle of least privilege	Can any actor reach a capability, route, field, or record beyond what their role entitles them to?	**Strong.** No authorization bypass found. One session-revocation defect (F1) and one timing oracle (F5).
Data handling safety	Is sensitive data — PII, credentials, signature blobs, tokens — over-collected, over-exposed, over-retained, or leaked into logs, URLs, or client bundles?	**Good, with a large and deliberately accepted public surface.** No unintended leak found; one log-transport edge case (U5).
Input validation	Is every trust boundary validated, and does malformed or hostile input fail safely?	**Good at the boundary, weaker on resource bounds.** Schema coverage is thorough, but three findings (F2, F3, F4) are about *unbounded or uncharacterized* input rather than unvalidated input.

3. Findings summary

ID	Finding	Severity	CWE	OWASP 2021	Lens	Location
F1	Self-service password change does not revoke live sessions — **remediated 2026-08-05**	**Medium**	CWE-613	A07 Identification & Authentication Failures	Least privilege	`src/modules/users/users.service.ts:71-74`
F2	Non-Latin-1 character in a receipt party field permanently breaks that receipt's public PDF	**Medium**	CWE-248	A04 Insecure Design	Input validation	`src/modules/receipts/hand-receipt.ts:279-281`
F3	Receipt signature accepts a 5 MB unvalidated PNG, decoded on a public route	**Medium**	CWE-409	A04 Insecure Design	Input validation	`src/modules/transfers/transfers.schema.ts:67-70`
F4	Unbounded per-id query fan-out from the `?items=` querystring	**Low**	CWE-770	A04 Insecure Design	Input validation	`src/app/receipts/new/page.tsx:13-16, :33`
F5	Login response time discloses whether an account exists	**Low**	CWE-208	A07 Identification & Authentication Failures	Data handling safety	`src/auth.ts:35-37`

4. Detailed findings

F1 — Self-service password change does not revoke live sessions

Severity: Medium · CWE-613 Insufficient Session Expiration · A07:2021 · Lens: least privilege · **Verified: CONFIRMED**

> **REMEDIATED 2026-08-05**, after this assessment was written. `changeUserPassword` now stamps `passwordChangedAt` in the same update, `changePasswordAction` signs the caller out to `/login?passwordChanged=1`, and the login page explains the sign-out. Two tests in `users.service.test.ts` pin it: the stamp on the success path, and its *absence* on the wrong-current-password path (stamping on a refusal would let anyone who can reach the form log the real owner out). `docs/SECURITY.md:57` has been corrected — it now states that all three password-mutation paths stamp the column, and records that this one did not until today. The finding is retained in full below as the record of what was found; the description is of the code **as assessed**, not as it now stands.

Location: `src/modules/users/users.service.ts:71-74`

```
await prisma.user.update({
  where: { id },
  data: { passwordHash: await hashPassword(newPassword) },
});
```

Failure scenario

1. An attacker obtains a victim's live session JWT — a shared or unlocked workstation, an exported cookie, a stolen laptop. Sessions are stateless Auth.js JWTs with no server-side revocation list.
2. The victim notices and does the one thing the product offers: `/account` → Change password.
3. `changePasswordAction` (`src/app/actions/account.ts:26`) verifies the current password and calls `changeUserPassword`, which writes `passwordHash` **and nothing else**.
4. The application's only revocation lever is `User.passwordChangedAt`, compared in the `jwt` callback at `src/auth.ts:116-130`. It is unchanged, so the comparison is false and the callback returns the token intact.
5. `requireUser()` re-reads only `role` and `isActive` (`src/lib/authz.ts:32-37`) — both unchanged — so authorization cannot compensate.
6. The attacker retains full role-appropriate access, including all of `/admin/*` if the victim is an ADMIN, until the 10-hour absolute bound from the *victim's original sign-in* expires, refreshing idle time freely within it.

Why Medium and not Low. It requires a pre-existing stolen session, and this app's short session policy caps the residual window at ~10 hours. It is rated Medium because it silently defeats *the exact remediation an incident responder would perform*, and because the security documentation asserts the opposite as fact — so the failure is invisible precisely when it matters.

Independent verification performed for this report. `passwordChangedAt` has exactly two writers repo-wide: `src/lib/password-reset.ts:46` (admin-initiated reset) and `src/modules/users/users.service.ts:47` (deactivation). Both stamp it correctly; the self-service change path is the sole omission. `signOut` appears once in application code (`src/app/actions/auth.ts:277` , the logout action) and is never called on this path. `docs/SECURITY.md:57` was read directly and does state: **"A password change revokes existing sessions."**

Remediation

```
data: { passwordHash: await hashPassword(newPassword), passwordChangedAt: new Date() },
```

No schema or callback change is needed — the revocation machinery at `src/auth.ts:106-137` is correct and would fire. Because this also revokes the caller's own token on its next request, `changePasswordAction` should redirect to `/login` with an explicit "password changed, sign in again" message rather than dropping the user into a silent logout. Add a unit test beside `src/modules/users/users.service.test.ts:59`, which already pins the equivalent assertion for deactivation, and correct `docs/SECURITY.md:57`.

F2 — Any non-WinAnsi character in a receipt party field permanently breaks that receipt's public PDF

Severity: Medium · CWE-248 Uncaught Exception · A04:2021 · Lens: input validation · **Verified: CONFIRMED**

Location: `src/modules/receipts/hand-receipt.ts:279-281` (also `:263`, `:265`, `:304`; fallback at `:169`)

```
for (const line of partyBlock(party)) {
  page.drawText(line, { x: 66, y, size: 11, font: helv, color: ink });
  y -= 15;
}
```

Failure scenario

1. A technician creates a hand receipt for a recipient named, for example, `Kalei‘okalani` (U+02BB ‘okina) or `Nguyễn` (U+1EC5). `partySchema` (`src/modules/transfers/transfers.schema.ts:21-47`) validates only `.trim().min(1)` — no charset restriction.
2. The receipt commits. `createReceiptAction` renders the PDF for the notification email inside its own try/catch (`src/app/actions/receipts.ts:99-100`) and only `console.error`s on failure — so the receipt is created and the emails go out, just without the attachment. **Nothing surfaces to the operator.**
3. Later, any visitor — the recipient following the emailed link, a PIN holder, or a technician — requests `GET /receipts/<n>/pdf`.
4. `partyBlock` emits the raw name into `page.drawText` with `helv = StandardFonts.Helvetica` (`:60`). pdf-lib routes standard-font text through WinAnsi, which **throws** on any code point outside cp1252. This was verified against the *installed* dependency: `node_modules/@pdf-lib/standard-fonts/dist/standard-fonts.js:6994-7003` (`throw new Error(name + ' cannot encode ...')`), reached from `StandardFontEmbedder.encodeTextAsGlyphs`.
5. The route (`src/app/receipts/[receiptNumber]/pdf/route.ts:6-18`) has no try/catch, and Route Handlers have no error boundary, so it returns **500 — permanently**. Receipts are immutable; no application path rewrites `senderName` / `receiverName`.

Why this matters here. No attacker is required. Ordinary Hawaiian, Vietnamese, and Polish names trigger it, and this is a Hawaii ARNG property book. The failure is permanent, unrecoverable in-app, and destroys availability of the signed DA 2062 — the system's authoritative artifact.

Adversarial correction incorporated. The refuter verified that the `set()` AcroForm helper at `:69-96` *is* wrapped in try/catch, so this finding rests specifically on the **unwrapped custody-page draws**, not on form-field population.

Remediation — two changes, both needed:

- **(a) Make the text renderable.** Register `@pdf-lib/fontkit` and embed a Unicode TrueType font with `subset: true` for every `drawText` / `widthOfTextAtSize` carrying user data; or sanitize through a helper using

`Encodings.WinAnsi.canEncodeUnicodeCodePoint` before drawing.

- **(b) Independently, contain the blast radius.** Wrap `renderReceiptPdf` in `src/app/receipts/[receiptNumber]/pdf/route.ts` so any render failure returns a handled error with a server-side log rather than an unhandled 500. This is a five-minute mitigation available before the font work.

Related sink: the same class affects `src/modules/items/qr-sheet.ts:32-39` via `serialNumber`, which would break `/admin/items/qr-sheet/pdf` for an entire admin selection.

F3 — Receipt signature accepts a 5 MB unvalidated PNG that is decoded on the public PDF route

Severity: Medium · CWE-409 Improper Handling of Highly Compressed Data · A04:2021 · Lens: input validation · **Verified: CONFIRMED (mechanism corrected by adversarial pass)**

Location: `src/modules/transfers/transfers.schema.ts:67-70`; decoded at `src/modules/receipts/hand-receipt.ts:156` (fires first) and `:291`

```
    receiverSignature: z
      .string()
      .startsWith(SIGNATURE_PREFIX, "Recipient signature is required")
      .max(MAX_SIGNATURE_BYTES, "Signature is too large"),
```

Failure scenario

1. Any authenticated USER creates a receipt (`createReceiptAction` , `requireUser()` at `src/app/actions/receipts.ts:18`). `parseReceiptForm` lifts `receiverSignature` verbatim from the form (`src/app/actions/receipts.parse.ts:27`).
2. Validation is **prefix + length only**. The shared validator `signatureError` is never called on this path.
3. The attacker submits a tiny PNG whose IHDR declares enormous dimensions. **No compression ratio is required:** the installed `UPNG.decode._decompress` (`node_modules/@pdf-lib/upng/dist/upng.js:7015-7016`) allocates `new Uint8Array((bpl+1+interlace)*h)` directly from the attacker's IHDR — no dimension sanity check, no CRC verification, no comparison against actual IDAT size. A ~100-byte PNG declaring 12000×12000 RGBA requests ~576 MB.
4. Every subsequent `GET /receipts/<n>/pdf` — **unauthenticated**, reachable by any PIN holder or receipt-link holder — re-triggers it.
5. The `try/catch` at `:290-298` *does* catch the JS-level throw, so this is **not** a permanent 500. What it cannot catch is `_filterZero` (`upng.js:7119-7143`), an unconditional `for(y=0;y<h;y++)` loop over `bpl` bytes per row — O(width × height) work driven purely by IHDR — plus the page-commit of the allocated buffer. Result: multi-second CPU and hundreds of MB RSS per request, from a stored ~100-byte value.
6. The same authenticated account can then hammer the route unmetered, since the 300/min anti-scraping budget is anonymous-only by design (`src/proxy.ts:208`).

The validation inconsistency, verified independently for this report. The application has a shared signature validator, `signatureError` (`src/lib/signature.ts:5`), capping at `MAX_SIGNATURE_LEN = 250_000`. It is used on **three** paths — the account saved-signature action (`src/app/actions/account.ts:51`), the returns action (`src/app/actions/returns.ts:44`), and the saved-signature schema (`src/modules/signatures/signatures.schema.ts:10`). The **receipt path alone** skips it and uses `MAX_SIGNATURE_BYTES = 5_000_000` (`src/modules/transfers/transfers.schema.ts:5`) — **20× larger, on the**

one signature path whose output is publicly reachable. Note also that `src/modules/signatures/signatures.schema.ts:4-6` comments that saved signatures *"obey the same rule as every other signature in the app"*, which is inaccurate.

Remediation. Validate at write time, not render time. Route `receiverSignature` through the shared `signatureError` (or at minimum lower `MAX_SIGNATURE_BYTES` to `MAX_SIGNATURE_LEN`), and extend `src/lib/signature.ts` to base64-decode the payload, check PNG magic bytes, and read IHDR width/height — rejecting anything whose pixel count exceeds a signature-sized bound (a drawn signature is a few hundred pixels tall). Placing it in `signatureError` means all four entry points inherit it and the documented invariant becomes true.

F4 — Unbounded per-id query fan-out on `/receipts/new` from the `?items=` querystring

Severity: Low · CWE-770 Allocation of Resources Without Limits or Throttling · A04:2021 · Lens: input validation · **Verified: CONFIRMED**

Location: `src/app/receipts/new/page.tsx:13-16` and `:33`

```
const ids = (itemsParam ?? "").split(",").map((s) => s.trim()).filter(Boolean);
if (ids.length === 0) notFound();

const loaded = (await Promise.all(ids.map((id) => getItem(id)))).filter(...)
```

Failure scenario

1. Any authenticated USER requests `/receipts/new?items=<id>,<id>,<id>,…` with the querystring filled toward the ~16 KB header cap.
2. Line 16 issues one `prisma.item.findUnique` per id, all concurrent. Line 33 issues a **second** concurrent wave — `getLastReceiver` per surviving item, each of which is `getHoldingTransfer`: a `findFirst` with a nested `some` over `lines.items`, an `orderBy`, and a filtered `include`, with **no scalar** `select` — so it materializes the whole `Transfer` row including the signature blob.
3. There is no dedupe, so repeating one known-good id N times produces N full holder lookups — roughly 600 real ids and ~2,000 statements per single HTTP GET.
4. The `MAX_RECEIPT_ROWS` / `MAX_ITEMS_PER_ROW` guards at `:20-21` run *after* both waves, so they bound what is **persisted**, never what is **queried**.
5. No rate limit applies: the proxy's 300/min budget is explicitly anonymous-only (`src/proxy.ts:208`).

This is the `Promise.all(ids.map(id => prisma...))` pattern that `CLAUDE.md` bans by name, in a security-relevant position.

Why Low. Authenticated-only, fully attributable, and bounded by URL length and a 10-connection pg pool. The realistic harm is a trusted technician degrading their own team's internal tool.

Remediation. Cap and dedupe *before* any query — `[...new Set(ids)].slice(0, MAX_RECEIPT_ROWS * MAX_ITEMS_PER_ROW)` — then replace both fan-outs with the batched helpers that already exist: `getItemsByIds` (`src/modules/items/items.service.ts:51`, already used by the qr-sheet route for this exact input shape) and `holdersForItems` (`src/modules/transfers/holders.query.ts:32`, which resolves custody for a whole page in one statement and selects only `receiverName`).

F5 — Login response time discloses whether an account exists

Severity: Low · CWE-208 Observable Timing Discrepancy · A07:2021 · Lens: data handling safety · **Verified: CONFIRMED**

Location: `src/auth.ts:35–37`

```
const user = await prisma.user.findUnique({ where: { email } });
if (!user || !user.isActive) return null;
if (!(await verifyPassword(password, user.passwordHash))) return null;
```

Failure scenario

1. An unauthenticated caller submits a login for a candidate address with any password.
2. For an unknown or deactivated account, `authorize` returns at line 36 after one indexed `findUnique`. For a live account, it proceeds to `verifyPassword` — bcrypt at cost 12, roughly 200–400 ms.
3. Both branches return the byte-identical string `"Invalid email or password."` (`src/app/actions/auth.ts:247` , `:265`) and both call `recordAuthFailure` , so content and side-effect count are symmetric. **Only the bcrypt term differs.**
4. Everything preceding the divergence is account-independent, including the Turnstile round trip, which runs at `:219` *before* `signIn()` at `:234` — so it is common-mode latency on both branches and averages out rather than masking the delta.
5. The attacker classifies a list of addresses as staff / non-staff, converting a blind spray into a targeted one. Because the email differs per probe, the narrow `(ip, email-hash)` bucket is fresh each time and only the 60/IP/15min ceiling binds.

Why Low. Yields account enumeration only, against a small admin-provisioned roster, and is throttled and CAPTCHA-gated. Real, but low yield.

Remediation. Equalize the work: when no active user is found, compare the submitted password against a fixed dummy bcrypt hash at the same cost before returning `null` , so both branches pay one compare. This mirrors treatment the reset surface already received deliberately — see `src/app/actions/auth.ts:307–343` , commented *"FIX #2 (timing side-channel)"*.

---

5. Principle of least privilege — assessment

Verdict: Strong. No authorization bypass was found.

Control	Evidence	Status
Every Server Action gates itself	49 of 49 call `requireUser()` / `requireAdmin()` as the first awaited statement	✅
Every Route Handler gates itself	6 of 6 (session, or constant-time secret compare for machine callers)	✅
Admin pages do not rely on inherited gates	10 of 10 call `requireAdmin()` directly	✅
Role and activation re-read per request	`src/lib/authz.ts:32-37` reads `role` + `isActive` from the DB, fails closed	✅
No client-supplied identity, role, or signature trusted	Verified across all 75 entry points	✅
Role-appropriate field restriction is server-side	`updateItemDetailsAction` picks the Zod schema by role; `z.object()` strips undeclared keys	✅
Derived-state writes reject non-settable values	`admin/actions/readiness.ts` — `DEPLOYED` / `IN_REPAIR` absent from the target enum, so a crafted POST is rejected rather than ignored	✅
Machine import endpoint	Bearer check *before* the body is read; `Content-Length` pre-check plus post-buffer backstop; generic errors; no delete or user-management capability	✅
Self-demotion guards	`src/app/admin/actions/users.ts:26, :37`	✅
Session revocation on credential change	**Two of three password-mutation paths stamp `passwordChangedAt`; self-service change does not**	❌ **F1**
Account existence disclosure	Timing delta on the login path	⚠️ **F5**

The capability-token grant (`src/lib/receipt-link-token.ts`) was examined specifically as a least-privilege question — *does the token admit its holder to more than the one receipt it names?* It does not. Attacks constructed and defeated: case-folding on the receipt number, encoding variants, path-segment manipulation against `RECEIPT_PATH` , and cross-receipt replay. `getTransferByReceiptNumber` uses `findUnique` on `receiptNumber.toUpperCase()` — exact match, **not** `mode:"insensitive"` — so the path segment, the signed value, and the selected row are the same string at every hop. The grant cookie is re-verified against the current path's receipt number, and `publicAccessAllowed()` is correctly **not** widened by holding a grant.

---

6. Data handling safety — assessment

Verdict: Good. No unintended exposure found. The large public surface is deliberate and signed off.

Concern	Finding
PII in list / search / type-ahead queries	✅ No signature blobs or PII pulled into list queries. `holdersForItems` selects only `receiverName`.
Signature blobs into client bundles	✅ `listReceiptsForItem` is Server-Component-only; blobs never enter the RSC payload.
Secrets in source	✅ No hardcoded live credentials, keys, tokens, or connection strings in application code.
Secrets in the client bundle	✅ None. No `NEXT_PUBLIC_` variable carries a secret.
Tokens in URLs / logs	⚠️ Reset token appears in access logs — **documented and accepted** (Known gap 3), mitigated by `Referrer-Policy` in `next.config.ts:10-16`. One undocumented log edge case — see **U5**.
Retention / purge	✅ 90-day closed-ticket purge and deactivated-account purge implemented; cron fails closed without `CRON_SECRET`.
Error messages	✅ Generic client-facing messages; detail logged server-side only.
Public surface exposure	⚠️ Large — **accepted by design.** See A1. Precision worth recording: the HTML receipt page exposes *less* than the PDF. `formatParty` (`src/modules/transfers/party.ts:4-8`) renders only isDcsim / name / rank / unit, while the PDF's `partyBlock` (`hand-receipt.ts:48-56`) additionally carries **contact number and email**, plus embedded signature images.

Hardcoded credentials — dedicated sweep result

No live hardcoded credentials exist in application code. Three items are worth recording:

1. **`prisma/seed-e2e.ts:10-12`** creates an ADMIN with a hardcoded password and, unlike its sibling `prisma/seed-analytics-demo.ts:33-56`, has **no target-database host allowlist**. Nothing invokes it automatically, so it requires operator error — but the project set this standard itself and did not apply it to the more dangerous script. Cheap fix. (**U6**)
2. **`docker-compose.yml`** binds `0.0.0.0` with `postgres/postgres` — local development only, referenced in no CI or deploy path; production is Supabase with separate credentials. Not a finding.
3. **`README.md:66`** **and** **`.env.example:57`** still advertise `admin@example.com / ChangeMe123!` as seed defaults. `prisma/seed.ts:24-29` now **throws** without `SEED_ADMIN_*`, so no such default exists today — but git history shows the default was introduced in `cfe582d` (2026-06-30) and removed in `5a82658` (2026-07-06), with the production deploy `ea27153` falling *between* those commits. **Action: check production for a legacy `admin@example.com` account.**

---

7. Input validation — assessment

Verdict: Thorough at the trust boundary. The gaps are about resource bounds, not missing schemas.

Injection class	Result
SQL injection	✅ None. All raw SQL is parameterized; sort keys come from a fixed allowlist and are guarded twice with `Object.hasOwn`.
Command injection	✅ None.
Code injection / `eval`	✅ None.
XSS	✅ No `dangerouslySetInnerHTML` anywhere in the tree.
Path traversal	✅ None.
SSRF	✅ None.
Open redirect	✅ None.
Prototype pollution	✅ None. The CSV parser maps headers through an explicit allowlist and builds rows by fixed field extraction.
Unsafe deserialization	✅ None.
ReDoS	✅ No user-controlled regex; `RECEIPT_PATH` is anchored and linear.

The residual input-validation risk is **unbounded input that is accepted, stored, and later processed on a public route** — F2 (uncharacterized charset), F3 (uncharacterized image dimensions), F4 (unbounded collection size). In each case a schema exists and runs; it simply does not constrain the dimension that matters.

---

8. Accepted gaps and accepted risks — register

Every item below is a **conscious acceptance, not a finding**. It is enumerated here so a reader can distinguish *"we looked at this and the team accepted it"* from *"we never looked."*

#	Accepted gap	Where it lives	Residual exposure, stated plainly	Recorded where	Implemented as documented?	What should force a re-decision
A1	Public, enumerable receipts and item pages	`src/proxy.ts:104–105`; `app/receipts/[receiptNumber]/page.tsx`; `app/i/[itemId]/page.tsx`; `app/receipts/[receiptNumber]/pdf/route.ts:6–18`	Anyone past the shared PIN reads the entire device catalog (serials, home unit, current holder, receipt history) and every hand receipt. The PDF additionally exposes contact number, email, and signature images that the HTML page does not	`CLAUDE.md:70`; `docs/SECURITY.md:282–286`, Known gap 2 (`:1350`)	✅ **Yes**	A new field added to any public query or page; any new route under `/receipts/*` or `/i/*`; serving external non-org parties
A2	Sequential `HR-000001…` receipt numbers	`src/modules/transfers/transfers.service.ts:45–46`	Receipt identifiers are trivially guessable; combined with A1, the whole corpus is walkable	`CLAUDE.md:70`; `docs/SECURITY.md:282–286`	✅ **Yes**	Receipts carrying data not already accepted as public; volume growth making bulk harvest materially worse than targeted lookup
A3	Shared 8-digit PIN; rotation is non-retroactive	Gate `src/proxy.ts:228–343`; storage `src/lib/public-access.ts:16–28` (bcrypt); setter `admin/actions/public-access.ts:7–18`	One secret for everyone, no per-person attribution; rotating it does not retire live unlock cookies (≤12 h lag). **Additionally: no weak-PIN policy** — the schema is `/^\d{8}$/` only, so `00000000` is accepted	`CLAUDE.md:72`; `docs/SECURITY.md:288–291`, Known gap 4 (`:1358`). **Weak-PIN sub-case is UNDOCUMENTED**	⚠️ **Deviates (minor).** Gate is exactly as documented; the *stated rationale* at `rate-limit.ts:116–117` ("20 guesses against 10^8 is hopeless") silently assumes a randomly-chosen PIN that nothing enforces	Gating anything beyond `/i/*` and `/receipts/*`; a move to per-person credentials; adopting a memorable-PIN convention
A4	Per-receipt capability token bypasses the PIN; never expires; no per-receipt revocation	`src/lib/receipt-link-token.ts` (whole file); proxy check `src/proxy.ts:262–302`; minted `src/modules/items/qr.ts:21–31`	Anyone holding the link — forwarded email, photographed QR, or harvested from a PDF by a PIN holder — reads that receipt permanently. The only revocation lever is rotating `AUTH_SECRET`, which also kills every session, every unlock cookie, and every QR already printed on paper	`CLAUDE.md:76`; `docs/SECURITY.md:382–442`, Known gap 12 (`:1491`); `DEPLOY.md:60`	✅ **Yes** — crypto and scope scrutinized specifically; every attack constructed failed closed	Any extension to `/i/*` or a broader grant; adding a per-receipt salt (which would make revocation possible and change the calculus); a reported leaked link
A5	RLS is not the authorization boundary; Prisma connects on a privileged bypassing role	`prisma/schema.prisma:523`; `prisma/manual/2026-07-20_lockdown_anon_grants.sql:18–20`; `src/lib/prisma.ts:7`	If the app layer has an authz hole, the database will not catch it. Nothing scopes rows for you	`CLAUDE.md:120–121`; `docs/SECURITY.md:841–856`	⚠️ **Cannot fully verify — see U3.** App-layer half confirmed (no Supabase client, no anon key in the tree). The deny-all RLS credited to an `rls_auto_enable` event trigger **has no definition anywhere in version control**	Any Supabase client or anon key entering the app; the Data API being enabled; any environment rebuilt from `prisma migrate deploy`

#	Accepted gap	Where it lives	Residual exposure, stated plainly	Recorded where	Implemented as documented?	What should force a re-decision
A6	Rate limiting fails **open** on a store error	`src/lib/rate-limit.ts:536–542` (consume), `:561–567` (read)	A Redis outage removes every rate limit app-wide — sign-in brute force, PIN guessing, and anti-scraping all become unmetered until the store returns. Logged, not alerted	`CLAUDE.md:109`; `docs/SECURITY.md:1158–1161`, Known gap 1 (`:1329`)	✅ **Yes.** Verified spend-before-work, refund only on identity-keyed success, narrow bucket before shared ceiling, `resetRateLimit` never on a refusal path	A shift to internet-facing or higher-value data; the store becoming reliable enough that failing closed is affordable; any control coming to *depend* on the limiter rather than treat it as friction
A7	`Item.currentUserEmail` is not email-validated	`src/modules/items/items.schema.ts:193, :216`	The column holds arbitrary strings like `"SGT Smith"` from CSV import. Anything treating it as a deliverable address will fail	`CLAUDE.md:59`	✅ **Yes**	Any code path that *emails* this field, or uses it as an identity or join key
A8	`mdm-import@service.invalid` non-loginable service account	`src/modules/items/import-actor.ts:7`; seeded `prisma/migrations/20260730000000_import_service_account/migration.sql:24`	An ordinary `User` row to the rest of the app — it appears in the admin Users list and `toggleUserActiveAction` will stamp `deactivatedAt` on it like any other. Until it has authored one `ImportBatch`, it is purgeable after 3 months	`CLAUDE.md:68`; `docs/SECURITY.md:245–276`	✅ **Yes.** Verified `isActive:false` is what blocks authentication and that `purgeDeactivatedUsers` counts `importBatches` (`account-purge.service.ts:31`)	A second service account being added; the row being deactivated before its first import in a fresh environment
A9	`POST /api/auth/callback/*` closed with 404	`src/proxy.ts:396–401`	None — this removes a surface. Residual: a future OAuth provider would need it reopened, and reopening without re-implementing Turnstile + the composite bucket + the velocity counter restores the bypass	`CLAUDE.md:98`; `docs/SECURITY.md:985–992`	✅ **Yes.** `GET` remains allowed for a future provider, as stated	Adding any OAuth or email provider to Auth.js
A10	Workstation holds a Vercel token + deploy hook that can ship production unattended	`scripts/gmail-token-rotation/`; handler `WindowsIntegration.psm1:577–601`; prod write `rotate-gmail-token.ps1:330–349`	Anyone with that Windows user's live session can trigger a production deploy, and a successful rotation ships `main` with nobody watching — colliding with the migrate-before-push rule	`docs/SECURITY.md:650–698`, Known gap 0c (`:1277`); `DEPLOY.md`	✅ **Yes.** The `%1` omission in the protocol handler is real and correct — no caller-supplied text reaches the command line	Either documented exit being taken (publishing the consent screen, or Workspace domain-wide delegation); the token being scoped beyond one project
A11	`MDM_IMPORT_SECRET` holder can create/update inventory; route unmetered and unlogged	`src/app/api/items/import/route.ts:51`; proxy exclusion `src/proxy.ts:466`	One shared secret, no per-caller identity, no way to revoke one holder. Rejected guesses leave no trace. Bounded by `MAX_IMPORT_ROWS` (2000), no delete/user/receipt capability, attributed to the service account	`docs/SECURITY.md` Known gaps 8 (`:1401`), 8a (`:1415`)	✅ **Yes.** The route is exemplary — bearer check before body read, dual size checks, generic errors	The endpoint gaining delete or user-management capability; more than one legitimate caller needing the secret

#	Accepted gap	Where it lives	Residual exposure, stated plainly	Recorded where	Implemented as documented?	What should force a re-decision
A12	No user-level non-repudiation; the seal is server-attested only	`src/lib/crypto.ts:29-51`; manifest `src/modules/transfers/seal.ts:18-33`	Anyone holding `SIGNING_PRIVATE_KEY` — Vercel env access, a compromised deploy, or the database plus that key — can mint a valid seal naming any `sealedByUserId`. In a genuine dispute, no verification rules this out	`docs/SECURITY.md:718-732`, Known gap 6 (`:1365`)	⚠️ **Deviates narrowly — see U2.** The stated property holds, but the manifest's *coverage* is narrower than §7's phrase "the record is unaltered since" implies	Any UI or briefing claiming non-repudiation; a real custody dispute; adoption of WebAuthn/PIV
A13	Most privileged mutations record no actor; no authentication event log	`src/app/admin/actions/items.ts:225-226, :249-250`; queue, timer, and user-management actions	*Who retired this device, who closed that ticket, who promoted this account* are unanswerable. No log of logins, failures, lockouts, resets, or PIN unlocks; no IP or user-agent capture anywhere	`docs/SECURITY.md` Known gap 7 (`:1383`)	⚠️ **Yes, but the register's list is incomplete** — it omits `deleteItemAction` (`src/app/admin/actions/items.ts:267-285`), the most destructive action in the app	Any compliance or audit requirement; a disputed deletion; the account-compromise claim the gap itself says cannot be corroborated
A14	Deleting an item on an open hand receipt is permitted	`src/app/admin/actions/items.ts:267-285`	The property-book row and its `/i/<id>` page vanish, so a printed QR on the physical device stops resolving; a later MDM import creates a fresh row with no carried-forward history. Mitigation is UI-only — nothing server-side refuses or logs it	`docs/SECURITY.md` Known gap 11 (`:1472`)	✅ **Yes**	Any requirement to preserve item-side custody history
A15	Custody email CC list is visible to all recipients and ships as code defaults	`src/lib/email-recipients.ts:35-38`	Every party on a receipt learns every other address, including an `army.mil` records mailbox. Editing the array changes who receives receipt PII with no config change and no deploy-time signal	`docs/SECURITY.md:614-648`, Known gap 10 (`:1454`)	✅ **Yes**	Receipts between two outside parties becoming common; any privacy requirement on party addresses
A16	A dead Gmail refresh token stops all outbound mail, with no fallback	`src/lib/email.ts:49-61`; `src/lib/gmail-oauth-email.ts:223-241`	Mail stops entirely, weekly, until a human re-mints the token	`docs/SECURITY.md:576-602`, Known gap 9 (`:1433`)	⚠️ **Deviates — see U5.** The *documented* failure mode is correct. A **partial** `GMAIL_*` set is a different, undocumented mode: `gmailOAuthConfigFromEnv` returns null and, with Resend unset, the app silently selects `LogEmailSender`, which prints full message bodies — including raw reset tokens and `?k=` capability tokens — to the platform log	Either 0c exit being taken; any log drain widening the log audience

Adjudication summary

- **Implemented exactly as documented (11):** A1, A2, A4, A6, A7, A8, A9, A10, A11, A14, A15.
- **Deviates (3):** A3 (documented brute-force rationale assumes randomness nothing enforces), A12 (seal coverage narrower than the wording), A16 (partial-config mode undocumented and leaks tokens to logs).
- **Cannot fully verify (1):** A5 — app-layer half confirmed; database-layer half rests on a trigger absent from version control and a live database not accessible to this assessment.
- **Register incomplete (1):** A13 omits `deleteItemAction`.

9. Reconciliation against the project's own risk register

`docs/SECURITY.md` maintains a "Known gaps & accepted risks" register at `:1273–1526`. It was reconciled line-by-line against the code.

9.1 Documented and still true (15 of 16 entries)

Entry	Register line	Confirmed at
0c — Workstation Vercel token / unattended deploy	`:1277`	`WindowsIntegration.psm1:589`
0a — Cloudflare-blocked visitor cannot sign in	`:1298`	`src/lib/turnstile.ts`, `components/TurnstileWidget.tsx`
0 — Bot defences config-gated; UA filter spoofable	`:1311`	`looksAutomated`, `src/proxy.ts:173–177`
1 — Rate limiting fails open; IP identifier forgeable	`:1329`	`src/lib/rate-limit.ts:536–542, :561–567`
2 — Public receipts/items enumerable	`:1350`	`src/proxy.ts:104–105`
3 — Reset token in access logs	`:1354`	`src/app/actions/auth.ts:337`; mitigation `next.config.ts:10–16`
4 — Shared PIN, non-retroactive rotation	`:1358`	`public-access-cookie.ts:100–110`
5 — JWT freshness costs one DB read per request	`:1362`	`src/lib/authz.ts:32–35` + `src/auth.ts:108`
6 — No user-level non-repudiation	`:1365`	`src/lib/crypto.ts`
7 — Unattributed mutations, no auth event log	`:1383`	`admin/actions/items.ts:225–226, :249–250`
8 — `MDM_IMPORT_SECRET` holder can write inventory	`:1401`	`api/items/import/route.ts:51`
8a — Import route unmetered/unlogged	`:1415`	`src/proxy.ts:466`
10 — CC list discloses addresses	`:1454`	`src/lib/email-recipients.ts:35–38`
11 — Deleting a signed-out item permitted	`:1472`	`admin/actions/items.ts:267–285`
12 — Receipt link never expires, no revocation	`:1491`	`src/lib/receipt-link-token.ts`

9.2 Documented but no longer true — stale or inaccurate claims

No register entry is fully stale — all sixteen still describe live code. That is a good result for a register of this size. However, **inaccurate claims elsewhere in the security documentation** cause the same harm:

1. `docs/SECURITY.md:57` **— *"A password change revokes existing sessions."* False** for the self-service path. This is **F1**, and it is the most consequential documentation inaccuracy in the tree because it misdirects incident response. *Verified directly for this report.*
2. `docs/SECURITY.md:847–848` **and** `prisma/schema.prisma:523` **— the** `rls_auto_enable` **event trigger.** Both credit a trigger with auto-enabling deny-all RLS, and `:848` cites `prisma/migrations/20260721170000_public_access_setting/migration.sql` as its home. **That file contains only a** `CREATE TABLE`**, an index, and an FK.** *Verified directly for this report:* the identifier appears in four places across `prisma/` — a comment, a `REVOKE EXECUTE` against it, a migration comment, and a schema comment — and a search for `CREATE EVENT TRIGGER`, `ENABLE ROW LEVEL SECURITY`, and `CREATE POLICY` across all of `prisma/` returns **nothing**. The trigger is asserted in three places and defined in none.
3. `docs/SECURITY.md:718` **§7 —** ***"the record is unaltered since."*** Overbroad. The manifest (`src/modules/transfers/seal.ts:18–33`) does not bind `qtyAuth`, `qtyIssued`, `unitOfIssue`, `lineNo`, `itemSummary`, or `createdAt` — all of which are printed on the DA 2062. Notably, the UI's *failure* string is already

correctly scoped ("a sealed field was altered", `ReceiptSealVerify.tsx:11`) while its *success* string is not ("the receipt is intact", `:10`).

4. `docs/SECURITY.md:230–232` lists "audits" as an admin-only capability, but the audit-signature *read* is `requireUser()` (`src/app/actions/audit.ts:11`). `CHANGELOG.md:291–297` records the staff-wide read as deliberate, so the **code is intentional and the §2 summary is the imprecise artifact**.
5. `README.md:66` / `.env.example:57` advertise seed defaults that no longer exist (see §6). **Action: check production for a legacy `admin@example.com`.**
6. `docs/SECURITY.md:235` cites `admin/actions/users.ts:24, :35` for the self-demotion guards; they are at `:26` and `:37`.
7. `src/app/i/[itemId]/page.tsx:286` cites a "100/min anti-scraping budget"; `API_POLICY` is **300/min** (`src/lib/rate-limit.ts:139`).
8. `src/modules/signatures/signatures.schema.ts:4–6` claims saved signatures "obey the same rule as every other signature in the app" — the receipt path does not (5 MB vs 250 KB). *Verified directly for this report.*
9. `CHANGELOG.md:292` describes `/i/<id>` as "already staff-only"; it is public behind the PIN gate — only the audit card within it is staff-gated.

9.3 Present in the code, absent from the register — the undocumented risks

These are the accepted-or-unnoticed risks that **nothing currently records**. Each is labeled as a deliberate-but-unrecorded tradeoff or an unnoticed defect.

#	Undocumented item	Location	Assessment
U1	Self-service password change does not revoke sessions	`src/modules/users/users.service.ts:71–74`	**Unnoticed defect.** Two of three password-mutation paths stamp the revocation column; this one does not, and the doc asserts the opposite. Reported as **F1** — **remediated 2026-08-05**, and `docs/SECURITY.md` now documents all three paths explicitly so the omission cannot recur silently.
U2	The Ed25519 seal does not cover the quantity columns or the printed date	`src/modules/transfers/seal.ts:18–33` vs `hand-receipt.ts:108, :126, :139–142`	**Deliberate-looking but unrecorded, and under-considered.** Correctly rejected as a *finding* (no application path writes those columns post-creation; `unitOfIssue` is the constant `"EA"`; quantities derive from the sealed serial list; `sealedAt` is sealed and surfaced) — but *what the seal proves* is a security property that belongs in §7 and is not there. Compounding it: `src/lib/crypto.ts` **is** on the `check-security-docs` watch list (`:88`) while `seal.ts` — the file defining what the signature binds — **is not**. The manifest can be narrowed without the guardrail firing.
U3	`rls_auto_enable` exists in no migration; the anon-grant lockdown is a one-shot manual `REVOKE`	`prisma/manual/2026-07-20_lockdown_anon_grants.sql:18–20`; absent from all 41 migrations	**Unnoticed infrastructure-as-code defect.** Rejected as a finding (no Supabase client, no anon key shipped, live DB unverifiable) — but **any environment rebuilt from `prisma migrate deploy` cannot reproduce the documented deny-all posture**, and two tables created *after* the lockdown — `PublicAccessSetting` (which holds the PIN's bcrypt hash) and `DeviceCategory` — were never covered by it. `prisma/` is also not on the watch list at all.
U4	No baseline security response headers	`next.config.ts:5–38` sets only `Referrer-Policy` on two path groups	**Unnoticed, but genuinely mitigated.** No CSP, `frame-ancestors`, `X-Frame-Options`, `X-Content-Type-Options`, or HSTS anywhere. Rejected unanimously because Auth.js defaults the session cookie to `SameSite=Lax` and the app never overrides it — a cross-origin iframe carries no session, so the framed app renders logged-out and clickjacking of admin controls fails. No `dangerouslySetInnerHTML` exists, so missing CSP has no demonstrated sink. Worth adding as defence in depth; not a live hole.

#	Undocumented item	Location	Assessment
U5	The log-only mail transport prints reset tokens and `?k=` capability tokens	`src/lib/email.ts:30`, reachable via partial `GMAIL_*` config (`gmail-oauth-email.ts:237-240`)	**Unnoticed defect, low impact.** Rejected because the log audience already holds `AUTH_SECRET` and `DATABASE_URL`, and the state is self-announcing (no mail is delivered at all). But §6 documents the log stub as a transport without noting it prints secrets.
U6	`prisma/seed-e2e.ts` creates an ADMIN with a hardcoded password and no target-database guard	`prisma/seed-e2e.ts:10-12`; `package.json:18`	**Unnoticed defect.** The sibling fixture `prisma/seed-analytics-demo.ts:33-56` implements exactly the guard this one lacks — a resolved-`DATABASE_URL` host allowlist — with a header comment explaining that guarding on `NODE_ENV` is not enough. **The project set its own standard and did not apply it to the more dangerous script.**
U7	No weak-PIN policy on the public-access gate	`src/app/admin/actions/public-access.ts:9`	**Unrecorded gap in a recorded control.** Known gap 4 records the shared-PIN and non-retroactive-rotation properties but not that trivially-guessable values are accepted. Rejected as a finding because a guessed PIN yields exactly the accepted-public surface — but the rationale at `rate-limit.ts:116-117` assumes randomness.
U8	`deleteItemAction` records no actor	`src/app/admin/actions/items.ts:267-285`	**Register incompleteness.** Known gap 7's enumeration omits the app's most destructive action (permanent, no undo).
U9	The `check-security-docs` guardrail does not watch itself, `purge-cron.yml`, `prisma/`, or `seal.ts`; its guard test never runs in CI	`scripts/check-security-docs.mjs:35-142, :170-173`; `.github/workflows/ci.yml` (jobs `sast`, `security-docs`, `build` — no vitest)	**Unnoticed process gap.** Rejected as a security finding (the actor must already hold write access, and the sanctioned `[skip security-doc]` bypass exists) — but **the one mechanism that keeps this documentation honest can be disarmed in a passing PR**, and `seal.ts` / `prisma/` escape it silently today. Given that documentation drift is this assessment's dominant theme, this is the highest-leverage process item in the report.
U10	The receipt signature path skips the shared validator and allows 20× the size	`src/modules/transfers/transfers.schema.ts:67-70` vs `src/lib/signature.ts:3`	**Unnoticed defect.** Reported as **F3**.
U11	`CRON_SECRET` / `MDM_IMPORT_SECRET` absent from `.env.example`	`.env.example`	**Template-completeness nit, not a risk.** Initially raised as "the retention purge silently never runs"; the word *silently* is wrong twice — `DEPLOY.md:165-167` documents the fail-closed behaviour verbatim, and `.github/workflows/purge-cron.yml:22-24, :28` hard-fails the scheduled run.

10. Coverage and limitations

Covered — all 506 tracked files accounted for

Area	Files	Treatment
`src/`	342 (228 non-test)	12 researchers across component × lens cells
— Proxy / PIN gate / receipt token	`proxy.ts`, `public-access*.ts`, `receipt-link-token.ts`, `web-hmac.ts`	Dedicated researcher + line-by-line read
— Auth / session / rate limiting / bot defence	`auth.ts`, `authz.ts`, `session-freshness.ts`, `rate-limit.ts`, `auth-velocity.ts`, `turnstile.ts`, `cron-auth.ts`	Dedicated researcher + line-by-line read
— Server Actions	49 across 22 files	Two researchers + full authorization-matrix sweep
— Route Handlers	6	All enumerated and gate-checked
— Pages / RSC / client components	22 pages + components	RSC payload boundaries traced
— Data layer, raw SQL, analytics	`modules/**`, `analytics.service.ts`	Two researchers; every `$queryRaw` traced
`prisma/`	47	Schema, all 41 migrations, manual DDL, all 3 seeds
`scripts/`	8	Including the PowerShell rotation tooling
`.github/workflows/`	2	Both, in full
`handoff/`, `public/`, root config	21	Secrets sweep + config researcher
`docs/`, `tests/`	75	Secrets sweep; docs read for reconciliation

Cross-cutting sweeps: a dangerous-sink sweep (SQL, command, code/eval, prototype pollution, path traversal, SSRF, open redirect, XSS, ReDoS, deserialization, DoS, prompt injection) and a complete authorization matrix across all 75 entry points.

Not covered — stated plainly

1. **Nothing was executed.** No build, no tests, no running server, no requests. Specifically unmeasured: the actual millisecond delta in **F5**, the exact memory threshold in **F3**, and whether F3's allocation surfaces as a catchable `RangeError` or a container kill.
2. **No live database or infrastructure access.** Could not check whether the `rls_auto_enable` trigger exists in production, whether the Supabase Data API is enabled, whether default anon privileges were restored on post-lockdown tables, or whether a legacy `admin@example.com` row survives. **Three items in this report end in "check production":** the RLS trigger, anon grants on `PublicAccessSetting` / `DeviceCategory`, and that legacy admin account.
3. **No deployed-environment configuration.** Could not verify which env vars are actually set in Vercel — whether Turnstile keys, the Upstash pair, or a complete `GMAIL_*` set are live is asserted from documentation, not observed.

4. **One framework behaviour left partly open.** Next 16 was confirmed to resolve Server Actions per-route rather than globally (from `.next/server/server-reference-manifest.json` and `node_modules/next/dist/server/app-render/manifests-singleton.js:150-183`), closing a proposed cross-route rate-limit bypass. One residual could not be ruled out: whether a Vercel deployment sets `__NEXT_PRIVATE_ORIGIN` such that the internal forwarded hop skips the middleware layer. Settling it requires a deployed build.
5. **Test files were read for secrets and pinned invariants, not audited as attack surface.**
6. **This is one nondeterministic static review.** It complements Semgrep, dependency scanning, and human code review; it does not replace them, and it is not a penetration test.

---

11. Checked and cleared

Twenty-one candidate issues were raised and **rejected by verification**. Recorded so this assessment documents what was examined and cleared, not only what failed.

#	Candidate	Why cleared
1	Any USER can read any admin's audit signature image	Killed by the adversarial pass: identical `Signature` blobs already reach *unauthenticated* visitors on public receipt pages/PDFs — accepted by design — so exposure to signed-in staff is a strictly smaller audience. The gate is a recorded decision (`CHANGELOG.md:291-297`), and harvesting confers no capability since a USER could already post any PNG.
2	Ed25519 seal omits quantities / printed date	Real coverage gap, but no application path writes those columns post-creation; requires direct DB write, already accepted. Recorded as **U2**.
3	`rls_auto_enable` absent from version control	Repo-state facts confirmed, but no attacker-reachable source: no Supabase client, no anon key in the tree, live DB unverifiable. Recorded as **U3**.
4	No CSP / `X-Frame-Options` / nosniff / HSTS	`SameSite=Lax` session cookie means a framed app renders logged-out; no `dangerouslySetInnerHTML` sink for CSP. Recorded as **U4**.
5	`listReceiptsForItem` unbounded, pulls signature blobs on a public page	Server-Component-only; blobs never enter the RSC payload. Row count not attacker-inflatable. Performance debt, not security.
6	`liveSearchAction` unmetered; cross-route action dispatch bypasses the proxy	Premise disproven: `/` is **not** matcher-excluded, so proxy gate 0b meters it; Next 16 resolves actions per-route. Both queries capped at 50 rows over accepted-public data.
7	Log-only mail transport prints reset and receipt tokens	Log audience already holds `AUTH_SECRET` and `DATABASE_URL` ; no route exposes logs; state is self-announcing. Recorded as **U5**.
8	`prisma/seed-e2e.ts` hardcoded ADMIN password, no guard	Not invoked by CI, Playwright, or tests — requires a maintainer to run it against a prod URL by hand. Recorded as **U6**.
9	`README.md` / `.env.example` publish `ChangeMe123!`	`prisma/seed.ts:24-29` throws without env vars; no live default exists. Stale docs + an unverifiable production-state question.
10	Analytics allowlist uses `in` instead of `Object.hasOwn`	Admin-only; inherited `Object.prototype` members are never `Prisma.Sql` , so they bind as parameters rather than splice. Worst case: an admin crashes their own dashboard.
11	`notifyPickupAction` unmetered, re-issues a capability token	The token is **deterministic**, not fresh per send — the same value already went to that party at receipt creation and is on the printed QR. Recipient address is DB-resolved.
12	`receiptSchema.itemIds` uncapped	Next's 1 MB action body cap applies; `createTransfer` throws `TOO_MANY_LINES` above 18 lines / 10 per row; first statement is a plain read that locks nothing.
13	No weak-PIN policy	A guessed PIN yields exactly the accepted-public surface. Recorded as **U7**.
14	GitHub Actions on mutable tags; `security-events: write` workflow-wide	`ci.yml` references **no secrets at all**; `pull_request` not `pull_request_target` ; fork SARIF upload skipped. No path to production.

#	Candidate	Why cleared
15	`purge-cron.yml` has no `permissions:` block	No `uses:` step, no checkout, no third-party action, never references `GITHUB_TOKEN`.
16	`docker-compose.yml` binds `0.0.0.0` with `postgres/postgres`	Local dev only; not referenced in CI or deploy; production is Supabase with separate credentials.
17	HKCU protocol handler can trigger a production deploy	`%1` deliberately omitted so no caller-supplied text reaches the command line; HKCU-scoped, so a same-user process could already run the script directly. Subsumed by accepted gap A10.
18	`Send-MdmImport.ps1` puts the bearer secret on `curl` argv	The secret already lives in the user's environment by the script's own guidance; same principals can read it either way. Payoff is accepted gap A11.
19	`check-security-docs.mjs` doesn't watch itself / workflows / `prisma/`	Documentation-currency guardrail, not a runtime control; requires write access, for which a sanctioned bypass already exists. Recorded as **U9**.
20	`searchReceiptsByNumber` doesn't escape LIKE `%` / `_`	Zero incremental capability — every receipt number is `HR-######`, so the ordinary literal query `HR-` already matches every row and returns the same capped 50.
21	Missing `CRON_SECRET` → purge silently never runs	Not silent: `DEPLOY.md:165-167` documents the fail-closed behaviour and `purge-cron.yml:22-24, :28` hard-fails the run. Recorded as **U11**.

12. Mitigation summary

Ordered by value per unit of effort. Nothing in this assessment modified the repository.

Priority 1 — Correctness of a security control ✅ DONE 2026-08-05

Item	Action	Effort	Control family
F1	~~Add `passwordChangedAt: new Date()` to the update in `users.service.ts:71–74`; redirect to `/login` with a "sign in again" message; add a unit test beside `users.service.test.ts:59`;~~ **~~correct~~ `docs/SECURITY.md:57` — all four completed 2026-08-05**	~15 min	`IA-5`, `AC-12`

This was the highest value-per-effort item in the report: a one-line data change that restores the app's only session-revocation lever on its most important path, plus a documentation correction that stops the security doc from misleading an incident responder. Both are done, along with the sign-out redirect and two regression tests.

Priority 2 — Availability of the authoritative artifact

Item	Action	Effort	Control family
F2 (b)	Wrap `renderReceiptPdf` in `receipts/[receiptNumber]/pdf/route.ts` in try/catch — handled error + server log instead of an unhandled 500	~5 min	`SI-11`
F2 (a)	Register `@pdf-lib/fontkit` and embed a subset Unicode TrueType font for all user-data draws, or sanitize via `Encodings.WinAnsi.canEncodeUnicodeCodePoint`. Apply to `qr-sheet.ts:32–39` as well	Half a day	`SI-10`
F3	Route `receiverSignature` through the shared `signatureError`; extend `src/lib/signature.ts` with a PNG magic-byte + IHDR dimension check so all four entry points inherit it	~1 hour	`SI-10`, `SC-5`

Doing F2(b) first buys containment immediately, before the font work lands.

Priority 3 — Infrastructure-as-code and guardrail integrity

Item	Action	Effort	Control family
U3	Move the `rls_auto_enable` DDL into a tracked migration; add `ALTER DEFAULT PRIVILEGES`; **then verify in production** that the trigger exists and that `PublicAccessSetting` / `DeviceCategory` carry no anon grants	~2 hours + a prod check	`CM-2`, `AC-3`
U9 / U2	Add `prisma/` and `src/modules/transfers/seal.ts` to the `check-security-docs` watch list; make the guardrail watch itself; run its guard test in CI	~30 min	`CM-3`, `AU-6`
§6.3	**Check production for a legacy `admin@example.com` account** from the 2026-06-30 → 07-06 window and remove it if present	~10 min	`IA-5`

Priority 4 — Documentation truth (the report's dominant theme)

Item	Action
§9.2	Correct the nine stale or inaccurate claims — most importantly `docs/SECURITY.md:57` (F1), `:847–848` (the undefined RLS trigger), and `:718` (seal coverage). Scope §7's success wording the way `ReceiptSealVerify.tsx:11` already scopes its failure wording
§8, §9.3	Add A3's weak-PIN sub-case, A12's seal-coverage narrowing, A16's partial-config log mode, and U8 (`deleteItemAction` unattributed) to the Known gaps register

Item	Action
§9.3	Record U2, U4, U5, U6, U7 as accepted risks or fix them — either is fine, but neither should remain unrecorded

Priority 5 — Hygiene and defence in depth

Item	Action
F4	Dedupe and cap `?items=` before querying; switch to `getItemsByIds` and `holdersForItems`
F5	Add a dummy bcrypt compare on the no-user branch of `authorize`, mirroring the reset surface's existing treatment
U4	Add CSP, `X-Frame-Options` / `frame-ancestors`, `X-Content-Type-Options`, and HSTS as defence in depth
U6	Add the `DATABASE_URL` host allowlist from `seed-analytics-demo.ts:33-56` to `seed-e2e.ts`
U11	Add `CRON_SECRET` and `MDM_IMPORT_SECRET` to `.env.example`

---

13. Conclusion

The application's security controls are, with one exception, correct and correctly reasoned. There is **no authorization bypass, no injection vector, and no unintended data exposure** in the assessed revision, and the most sensitive recent addition — the per-receipt capability token — withstood targeted cryptographic and scope attack. The large public surface is deliberate, documented, and explicitly accepted by the team; it is not a defect and was not treated as one.

The real risk this assessment surfaces is **documentation drift**. A security document that asserts a revocation which does not happen (F1 / `docs/SECURITY.md:57`), and a schema comment plus two doc sites that credit a database-level control defined nowhere in version control (U3), are more dangerous than most Medium findings, because they cause a responder or a future maintainer to rely on protection that is not there. That the project's own `check-security-docs` guardrail does not currently watch `prisma/` or `seal.ts` — and can be disarmed in a passing PR (U9) — is the structural reason drift went unnoticed, and fixing it is the single change most likely to keep this report's conclusions true a year from now.

Generated by the Claude Security plugin (multi-agent static review with adversarial verification). Static analysis only — no code executed, no live database or deployed environment accessed. Complements, and does not replace, Semgrep SAST, dependency scanning, human code review, and penetration testing.